

CAPÍTULO 1

LINGUAGENS, GRAMÁTICAS E RECONHECEDORES

ALFABETOS E LINGUAGENS

ALFABETO:

Um **alfabeto** (ou **vocabulário**) Σ é um conjunto finito, não vazio de símbolos.

Ex: Temos os alfabetos: $\{0, 1\}$ ou $\{a, b\}$

O alfabeto da língua portuguesa: $\{a, b, c, \dots, z\}$

O conjunto dos caracteres ASCII

Para cada aplicação específica o usuário deve escolher o melhor alfabeto:

Para exemplos em aula: $\{0, 1\}$ ou $\{a, b\}$

Para ensinar a linguagem Pascal, o alfabeto deverá ter símbolos como:
program, begin, if, then, else;

Para implementar a linguagem Pascal o alfabeto adequado conterá símbolos ASCII (letras, números, +, *, etc...)

PALAVRA, CADEIA DE CARACTERES OU SENTENÇA:

Uma **palavra** (ou **cadeia** ou **cadeia de caracteres**, ou **sentença**, ou **string**) sobre um alfabeto Σ é uma sequência finita de símbolos de Σ justapostos.

x é uma cadeia $\Rightarrow x = a_1 a_2 \dots a_n$, $n \geq 0$, onde $a_i \in \Sigma$, $i = 1, 2, n$

Se $n = 0 \Rightarrow \varepsilon$ (cadeia vazia), também representada por Λ é uma palavra sem símbolo.

Σ^* é o conjunto de todas as palavras possíveis sobre Σ

Σ^+ é o conjunto Σ^* sem a palavra vazia: $\Sigma^+ = \Sigma^* - \{\varepsilon\}$

TAMANHO OU COMPRIMENTO DE UMA CADEIA DE CARACTERES

Tamanho ou comprimento de uma cadeia x sobre um alfabeto Σ (representado por $|x|$) é o número total de símbolos de Σ que compõem a palavra x

$$\text{Ex.: } \Sigma = \{0, 1\} \Rightarrow \begin{cases} |011| = 3 & (\text{n.º de elementos na cadeia}) \\ |\epsilon| = 0 \end{cases}$$

PREFIXO, SUFIXO, SUBPALAVRA:

Prefixo de uma palavra é qualquer seqüência de símbolos inicial da palavra.

Sufixo de uma palavra é qualquer seqüência de símbolos final da palavra.

Subpalavra de uma palavra é qualquer seqüência de símbolos contígua da palavra.

EXEMPLOS: Prefixo, Sufixo, Subpalavra

- 1) $abcb$ é uma palavra sobre o alfabeto $\{a, b, c\}$
- 2) Se $\Sigma = \{a, b\}$, $\Sigma^+ = \{a, b, aa, ab, ba, bb, aaa, \dots\}$ e $\Sigma^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$
- 3) $|abcb| = 4$ e $|\epsilon| = 0$
- 4) $\epsilon, a, ab, abc, abcb$ são prefixos da palavra $abcb$ e $\epsilon, b, cb, bcb, abcb$ são os sufixos
- 5) Qualquer prefixo ou sufixo da palavra é uma subpalavra.

LINGUAGEM FORMAL

É um conjunto de palavras sobre um alfabeto

EXEMPLOS: Linguagem

- 1) O conjunto vazio e o conjunto formado pela palavra vazia são linguagens sobre Σ (sabemos que $\{\}$ $\neq$ $\{\epsilon\}$)
- 2) O conjunto de palíndromos (palavras que tem a mesma leitura da esquerda para direita e vice-versa) sobre Σ é um exemplo de linguagem infinita. Temos que ϵ , a , b , aa , aaa , aba , bab , bbb , $aaaa$, ... são palavras desta linguagem.

CONCATENAÇÃO:

Sejam x e y duas cadeias sobre Σ a concatenação de x e y é definida como a cadeia xy ou $x \circ y$

$$\text{Ex.: } \Sigma = \{0, 1\} \text{ , } x = 010 \text{ , } y = 10 \Rightarrow \begin{cases} xy = 01010 \\ yx = 10010 \end{cases}$$

OBS.1: A cadeia vazia é o elemento neutro da operação de concatenação
 $\Rightarrow x = \epsilon x = x$

OBS.2: Concatenação sucessiva: $(10)^n = \underbrace{101010 \dots 10}_n$
 n ocorrências concatenadas da cadeia 10

PRODUTO:

Sejam X e Y conjuntos de cadeias sobre Σ . O produto de X e Y é definido por:

$$XY = \{xy \mid x \in X, y \in Y\}$$

Notação: $X^0 = \{\varepsilon\}$

$$X^{i+1} = X^i X, i \geq 0$$

Operações com um conjunto X de cadeias:

Fechamento de Kleene (*Kleene's closure*): X^*

Fechamento Positivo (*positive closure*): X^+

$X^+ = \cup X^i, i \geq 1 \quad \therefore$ todas as concatenações possíveis

$X^* = \cup X^i, i \geq 0 \quad \therefore$ todas as concatenações possíveis, mais a cadeia vazia.

Exemplo: $\Sigma = \{0, 1\}$

$\Sigma^* = \{\varepsilon, 0, 1, 01, 10, 00, 11, \dots\}$

$\Sigma^+ = \{0, 1, 01, 10, 00, 11, \dots\}$

LINGUAGEM:

Uma linguagem L é um conjunto qualquer de cadeias sobre um alfabeto Σ , onde:

$$L \subset \Sigma^*.$$

Como representar L?

- Se L é finito, basta listar todas as cadeias.
- Se L é infinito, existem 2 sistemas principais para representação de linguagens:

1. O sistema gerador: **Gramática**

2. O sistema reconhecedor: **Autômato**

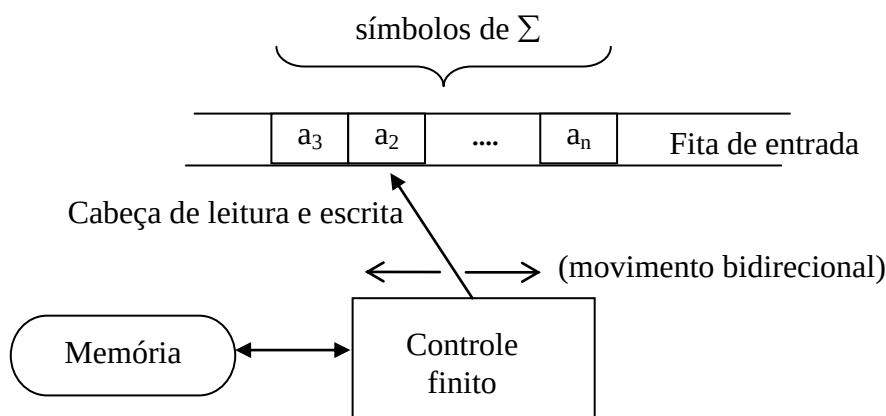
RECONHECEDORES E GRAMÁTICAS

RECONHECEDORES:

Um reconhecedor de **L** é a representação de um procedimento que, quando apresentado a uma cadeia qualquer:

- **pára** e responde **sim**, após um número finito de passos, caso a cadeia pertença a **L**; ou
- **pára** e responde **não**, caso a cadeia não pertença a **L**; ou
- **não pára**, caso a cadeia não pertença a **L**.

Simbolicamente:



Uma configuração do reconhecimento é uma descrição:

- (a) do estado do controle finito
- (b) do conteúdo da fita de entrada e da posição da cabeça
- (c) do conteúdo da memória.

Um reconhecedor aceita (ou reconhece) uma cadeia w se:

1. partindo de uma configuração inicial;
2. faz uma seqüência finita de movimentos; e
3. termina em uma configuração final.

A linguagem aceita por um reconhecedor R é:

$$L(R) = \{w \in \Sigma^* \mid R \text{ aceita } w\}$$

GRAMÁTICA:

Uma gramática é uma 4-upla $G = (V, T, P, S)$ onde:

V é um alfabeto de símbolos auxiliares, finito e não vazio, chamados de não-terminais

T é o alfabeto de símbolos, finito e não vazio, no qual a linguagem é definida, chamados de símbolos terminais ou terminais, tal que $T \cap V = \emptyset$ (T e V são disjuntos)

P é um conjunto finito de pares denominados regras de produção tal que a primeira componente é palavra de $(V \cup T)^+$ e a segunda componente é a palavra de $(V \cup T)^*$

S $\in V$, é o símbolo (não terminal) inicial ou variável inicial.

Cada regra de produção individual corresponde a um par de cadeias (α, β) . Em vez de $(\alpha, \beta) \in P$, a regra que permite a reescrita de α como β é escrita como $\alpha \rightarrow \beta$, onde:

$$\alpha \in (V \cup T)^* V (V \cup T)^* \quad \text{e} \quad \beta \in (V \cup T)^*$$

As regras de produção definem as condições de geração das palavras da linguagem. A derivação de uma palavra se dá pela aplicação de uma regra de produção. A aplicação sucessiva de regras de produção permite derivar as palavras da linguagem representada pela gramática.

Regras de produção com o mesmo lado esquerdo α tais como $\alpha \rightarrow \beta_1, \alpha \rightarrow \beta_2, \dots, \alpha \rightarrow \beta_n$ podem ser abreviadas como $\alpha \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$

Ex: $G = (V, T, P, S)$ onde:

$$V = \{A, S\}$$

$$T = \{a, b\}$$

$$P = \{S \rightarrow ab,$$

$$S \rightarrow aASb,$$

$$S \rightarrow bSb,$$

$$AS \rightarrow bSb,$$

$$A \rightarrow \varepsilon$$

$$aASAb \rightarrow aa \}$$

Notação:

Letra maiúscula: símbolos não-terminais, variáveis

Letra minúscula: símbolos terminais

$\alpha, \beta, \dots$: cadeias quaisquer (variáveis ou terminais)

$x, y, z, \dots$: cadeias (palavras) de terminais

$V, N, \Sigma \dots$: alfabetos

Exemplo de cadeias geradas por essa gramática:

1. $S \rightarrow ab$
2. $S \rightarrow aASb \rightarrow abSbb \rightarrow ababbb$

Ou seja, a linguagem gerada $L(G) = \{ab, ababbb, \dots\}$

OBS:

- A linguagem gerada por G pode ser obtida pela construção da Árvore de Derivação correspondente.
- Se L é gerada por G , qualquer gramática G' obtida a partir de G passa a gerar L' , possivelmente diferente de L .
- Ao se projetar uma gramática que gere L , deve-se tomar o cuidado de permitir a geração de todas e apenas as cadeias de L .

Ex: que gramática pode gerar $L = \{0^n 1^n \mid n \geq 1\} = \{01, 0011, 000111, \dots\}$?

$T = \{0, 1\}$

Símbolo inicial: S

$V = \{S\}$

$P = \{S \rightarrow 0S1, S \rightarrow 01\}$

DERIVAÇÃO:

Sejam: uma gramática $G = (V, T, P, S)$, $\alpha' \in (V \cup T)^+$, e $\beta' \in (V \cup T)^*$

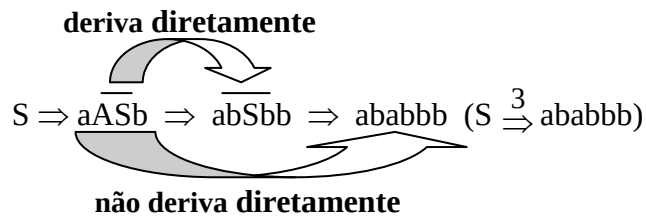
α' deriva diretamente β' ($\alpha' \Rightarrow \beta'$) se existem $\alpha_1, \alpha_2, \alpha, \beta \in (V \cup T)^*$ tais que:

$$\alpha' = \alpha_1 \alpha \alpha_2,$$

$$\beta' = \alpha_1 \beta \alpha_2 \text{ e}$$

$$\alpha \rightarrow \beta \in P \text{ ou } \alpha_1 \alpha \alpha_2 \Rightarrow \alpha_1 \beta \alpha_2$$

Por exemplo, segundo G definido anteriormente:



Notação: quando consideramos vários passos de uma derivação, usamos as definições de operações com relações:

$\Rightarrow^i$ deriva em i passos sucessivos

$\Rightarrow^*$ deriva em zero ou mais passos sucessivos

$\Rightarrow^+$ deriva em um ou mais passos sucessivos

Gramática é considerada um formalismo de geração, pois permite derivar (“gerar”) todas as palavras da linguagem que representa.

Ex: Seja a gramática:

$$G = (V, T, P, S) = (\{S\}, \{0, 1\}, \{(S, 0S1), (S, \epsilon)\}, S)$$

$$S \Rightarrow 0S1, \quad 0S1 \Rightarrow 00S11, \quad 00S11 \Rightarrow 000S111, \quad 000S111 \Rightarrow 000111$$

$$\text{ou na forma compacta: } S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000S111 \Rightarrow 000111$$

$$\text{ou ainda: } S \Rightarrow^4 000111 \text{ e } S \Rightarrow^* 000111$$

as cadeias: $S, 0S1, 00S11, 000S111, 000111$ são formas sentenciais da gramática.

a mais importante é 000111 , composta só por terminais e também conhecida como "palavra", que fará parte da linguagem da gramática, segundo a próxima definição.

FORMAS SENTENCIAIS:

Sejam $G = (V, T, P, S)$. O conjunto de Formas Sentenciais de G é definido por:

$$S(G) = \{ \alpha \in (V \cup T)^* \mid S \Rightarrow^* \alpha \}$$

LINGUAGEM GERADA:

Sejam $G = (V, T, P, S)$. A Linguagem Gerada (ou Representada) pela gramática G , denotada por $L(G)$ ou $GERA(G)$, é o conjunto composto por todas as palavras de símbolos (ou cadeias) terminais deriváveis a partir do símbolo inicial S , usando as regras de P , ou seja:

$$L(G) = \{ w \in T^* \mid S \Rightarrow^+ w \} = T^* \cap S(G)$$

OBS: As formas sentenciais de uma gramática são as próprias árvores de derivação associadas à gramática, enquanto a linguagem gerada constitui o conjunto das folhas da árvore.

OBS: No contexto usual de linguagens de programação,

1. T (símbolos) $\equiv$ palavras reservadas + variáveis definidas + símbolos numéricos + operadores + delimitadores...
2. cadeia $\equiv$ programa sintaticamente correto
3. L (linguagem) $\equiv$ conjunto de programas sintaticamente corretos
4. G (gramática) $\equiv$ estrutura sintática

EXEMPLO: Gramática, Derivação, Linguagem Gerada

Seja a gramática $G = (V, T, P, S)$ onde: $V = \{S, D\}$

$$T = \{0, 1, 2, \dots, 9\}$$

$$P = \{S \rightarrow D, S \rightarrow DS, D \rightarrow 0 \mid 1 \mid \dots \mid 9\}$$

Gera sintaticamente o conjunto dos números naturais, distingue 0 a esquerda (123 e 0123)

Uma derivação do número 243 é:

$$S \Rightarrow DS \Rightarrow 2S \Rightarrow 2DS \Rightarrow 24S \Rightarrow 24D \Rightarrow 243$$

Existe mais alguma derivação do número 243 ?

Logo temos: $S \Rightarrow^* 243$, $S \Rightarrow^+ 243$, $S \Rightarrow^6 243$

EXEMPLO: Gramática, Derivação, Linguagem Gerada

$G = (\{S, X, Y, A, B, F\}, \{a, b\}, P, S)$, onde:

$P = \{ S \rightarrow XY,$

$X \rightarrow XaA \mid XbB \mid F$

$Aa \rightarrow aA, Ab \rightarrow bA, AY \rightarrow Ya,$

$Ba \rightarrow aB, Bb \rightarrow bB, BY \rightarrow Yb,$

$Fa \rightarrow aF, Fb \rightarrow bF, FY \rightarrow \varepsilon\}$

gera a linguagem: $\{ww \mid w \text{ é palavra de } \{a, b\}^*\}$

Uma derivação para a palavra baba é:

$$\begin{aligned} S &\Rightarrow XY \Rightarrow XaAY \Rightarrow XaYa \Rightarrow XbBaYa \Rightarrow XbaBYa \Rightarrow XbaYba \\ &\Rightarrow FbaYba \Rightarrow bFaYba \Rightarrow baFYba \Rightarrow baba \end{aligned}$$

Existe mais alguma derivação para a palavra baba?

GRAMÁTICAS EQUIVALENTES:

Duas gramáticas G_1 e G_2 são chamadas de Gramáticas Equivalentes se, e somente se, ambas definem a mesma linguagem: $L(G_1) = L(G_2)$ ou $GERA(G_1) = GERA(G_2)$

HIERARQUIA DE CHOMSKY

A gramática, como foi definida anteriormente, é chamada de gramática **Irrestrita** ou **Tipo-0** (ou ainda: gramática Semi-Thue; ou gramática de Estrutura de Frase).

GRAMÁTICA SENSÍVEL AO CONTEXTO OU TIPO-1:

$G = (V, T, P, S)$ é gramática **Sensível ao Contexto** ou **Tipo-1**, se toda produção de P é da forma:

$$1) \alpha A \gamma \rightarrow \alpha \beta \gamma \quad \left\{ \begin{array}{l} A \in V \\ \alpha, \gamma \in (V \cup T)^* \\ \beta \in (V \cup T)^+ \end{array} \right.$$

2) $S \rightarrow \varepsilon$, sendo que se esta regra de produção ocorrer, S não pode aparecer no lado direito de qualquer outra produção.

GRAMÁTICA LIVRE DO CONTEXTO OU TIPO-2:

$G = (V, T, P, S)$ é gramática **Livre de Contexto** ou **Tipo-2** se toda produção de P é da forma:

$$A \rightarrow \alpha, \quad \begin{array}{l} A \in V \\ \alpha \in (V \cup T)^* \end{array}$$

GRAMÁTICA REGULAR OU TIPO-3:

$G = (V, T, P, S)$ é uma gramática **Regular** ou **Tipo-3**, se toda produção de P é da forma linear à direita (*right-linear*):

$$\begin{array}{ll} 1) A \rightarrow wB & \left\{ \begin{array}{l} A, B \in V \\ w \in T^* \end{array} \right. \\ 2) A \rightarrow w & \end{array}$$

Ou da forma linear à esquerda (*left-linear*):

$$\begin{array}{ll} 3) A \rightarrow Bw & \left\{ \begin{array}{l} A, B \in V \\ w \in T^* \end{array} \right. \\ 4) A \rightarrow w & \end{array}$$

GRAMÁTICA TIPO-X:

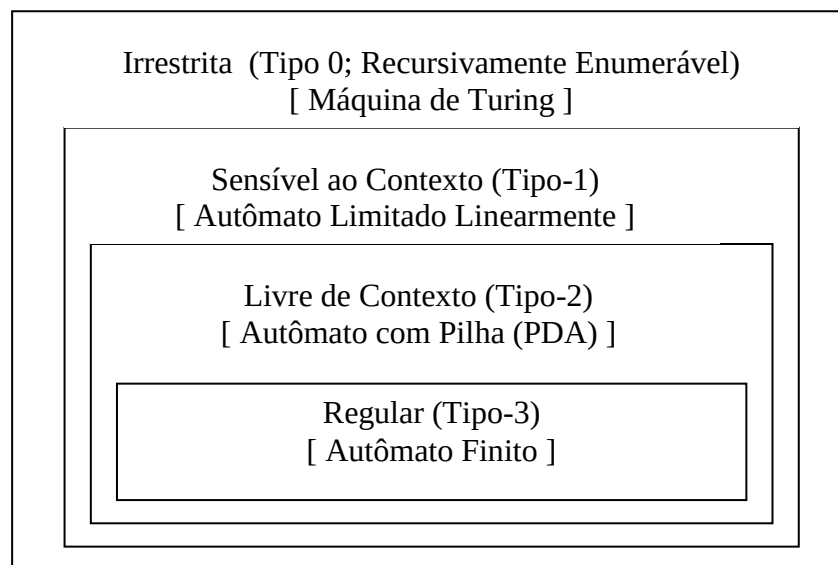
Uma linguagem L é do **Tipo-x**, se existe uma gramática **Tipo-x**, G , tal que $L = L(G)$ (isto é, a linguagem L é gerada pela gramática G).

OBS: Tipos 2 e 3: Só existe 1 não-terminal do lado esquerdo.

Tipo-1: Pode haver mais de 1 não-terminal do lado esquerdo, mas só 1 é transformado em cada regra de produção

Tipo-0: Qualquer quantidade de não-terminais do lado esquerdo das produções

Hierarquia de Chomsky: Relacionamento entre Gramáticas (ou Linguagens) e seus reconhecedores correspondentes.



OBS2: Segundo Wolfram:

Tipo-0: Produções arbitrárias.

M.T.: Memória arbitrariamente grande.

Tipo-1: Produções da forma $\alpha \rightarrow \beta$ tal que $|\alpha| \leq |\beta|$, $\beta, \alpha \in (V \cup T)^*$.

A.L.L.: Memória proporcional ao comprimento da cadeia de entrada.

Tipo-2: Produções da forma $A \rightarrow \alpha$ tal que $A \in V$, $\alpha \in (V \cup T)^*$, e $|\alpha|$ finito.

P.D.A.: Memória em pilha, com uma quantidade fixa de elementos disponíveis em um dado tempo.

Tipo-3: Produções da forma (unitária) $A \rightarrow sB$ ou $A \rightarrow s$ tal que $A, B \in V$, $s \in (T \cup \epsilon)$.

A.F.: Sem memória.